

ZeroShare

Privacy-First Personal Data Consent Management Platform

zs

Research Paper — Technical Documentation
July 2026

1. Abstract

ZeroShare is a privacy-first personal data consent management platform enabling individuals to store sensitive information in a secure encrypted vault and govern third-party access through a structured consent workflow. Built on a zero-trust architecture, it integrates a machine-learning risk-scoring microservice that evaluates consent requests in real time, transitioning from rule-based heuristics to logistic regression after sufficient labelled data is collected.

2. Problem Statement

Personal data is routinely collected and monetised by third-party applications without meaningful user control. Regulations such as GDPR (EU) and DPDPA (India) mandate consent but lack practical tooling for users to exercise those rights.

- Lack of visibility into who has access to personal data
- No centralised auditable consent ledger
- No risk signal before granting data access
- No mechanism to revoke access once granted
- No immutable audit trail of data governance decisions

3. Objectives

1. Secure personal data vault for sensitive information storage
2. Full consent lifecycle: Pending → Granted / Denied → Revoked
3. Real-time ML-based risk scoring of consent requests
4. Immutable audit log of all data governance events
5. Multi-role access control (User and Admin)
6. Email notifications and OTP-based two-factor authentication
7. Docker Compose for zero-setup onboarding and deployment

4. System Architecture

ZeroShare follows a four-layer microservices-inspired architecture deployed via Docker Compose, with each layer independently scalable and communicating over well-defined interfaces.

Layer	Technology	Role
Frontend	React 18 + Vite + Nginx	SPA dashboard on port 80
Backend API	Node.js + Express	REST API + WebSocket on port 5001
Database	PostgreSQL 16	Relational data store on port 5432
ML Service	Python 3.11 + FastAPI	Risk scoring microservice on port 8000

Technology Stack

Component	Technology	Purpose
UI Framework	React 18 + Vite	Single-page application

Component	Technology	Purpose
UI Components	shadcn/ui + Tailwind CSS	Design system
Animations	Framer Motion	Smooth UI transitions
HTTP Client	Axios	API communication
Real-time	WebSocket (ws)	Live push notifications
Backend	Node.js + Express	REST API server
Authentication	JWT (jsonwebtoken)	Stateless auth tokens
Password Hashing	bcrypt (rounds=10)	Secure password storage
Encryption	AES-256 (Node crypto)	Data vault encryption at rest
Email	Nodemailer + Gmail SMTP	Transactional emails
Database	PostgreSQL 16	Relational data store
ML Framework	FastAPI + Python 3.11	Risk scoring microservice
ML Algorithm	Logistic Regression	Consent risk classification
Security Headers	Helmet.js	HTTP security hardening
Containerisation	Docker + Docker Compose	Deployment and orchestration
Reverse Proxy	Nginx	Static serving + API proxy

5. Database Design

ZeroShare uses a normalised PostgreSQL schema with seven tables. All foreign keys use cascading deletes to ensure data integrity when users are removed.

Table: users

Column	Type / Constraint	Description
id	SERIAL PRIMARY KEY	Auto-increment primary key
name	VARCHAR(100)	Full name
email	VARCHAR(100) UNIQUE	Login email
password_hash	TEXT	bcrypt hash
role	VARCHAR(20) DEFAULT 'user'	'user' or 'admin'
avatar_url	VARCHAR(500)	Profile photo URL
two_fa_enabled	BOOLEAN	OTP 2FA toggle
is_suspended	BOOLEAN	Admin suspension flag

Table: consents

Column	Type / Constraint	Description
id	SERIAL PRIMARY KEY	Primary key
user_id	INTEGER FK -> users	Consent owner
app_name	VARCHAR(200)	Requester display name
data_type	VARCHAR(200)	What data is requested
purpose	TEXT	Stated reason for access
duration	VARCHAR(100)	e.g., "30 Days", "Permanent"
risk_level	VARCHAR(10)	'low' 'medium' 'high'
risk_score	INTEGER	0-100 ML score
status	VARCHAR(20)	PENDING GRANTED DENIED REVOKED
requester_type	VARCHAR(10)	'app' 'website'
requester_url	VARCHAR(500)	URL for website requesters
expires_at	TIMESTAMPTZ	Auto-expiry timestamp

Table: user_data

Column	Type / Constraint	Description
id	SERIAL PRIMARY KEY	Primary key
user_id	INTEGER FK -> users	Data owner
data_type	VARCHAR(100)	e.g., "Medical Record"
value	TEXT	Encrypted sensitive value
record_type	VARCHAR(10)	'text' 'file'
file_name	VARCHAR(255)	Original filename
file_size	INTEGER	File size in bytes
file_url	VARCHAR(500)	Server-side file path

Table: audit_logs

Column	Type / Constraint	Description
id	SERIAL PRIMARY KEY	Primary key
user_id	INTEGER FK -> users	Actor
event_type	VARCHAR(100)	e.g., "Consent Granted"
app_name	VARCHAR(200)	Related application
data_accessed	TEXT	Data scope
status	VARCHAR(50)	Success / Blocked

Table: notifications

Column	Type / Constraint	Description
id	SERIAL PRIMARY KEY	Primary key
user_id	INTEGER FK -> users	Recipient
title	VARCHAR(200)	Notification title
message	TEXT	Full message body
is_read	BOOLEAN	Read status

6. Backend API Design

Module	Method	Endpoint	Description
Auth	POST	/api/auth/register	Register + send welcome email
Auth	POST	/api/auth/login	Login + JWT + 2FA trigger
Auth	POST	/api/auth/verify-otp	OTP verification for 2FA
Auth	POST	/api/auth/forgot-password	Send password reset email
Auth	POST	/api/auth/reset-password	Token-based password reset
Consents	GET	/api/consents	List consents (paginated, sorted)
Consents	POST	/api/consents	Create consent (ML scored)
Consents	PATCH	/api/consents/:id/status	Grant / Deny / Revoke
Data Vault	GET	/api/data	List all data records
Data Vault	POST	/api/data	Add text entry
Data Vault	POST	/api/data/upload	Upload file (PDF, DOC, image)
Data Vault	GET	/api/data/:id/view	View file inline in browser
Data Vault	DELETE	/api/data/:id	Delete record + file from disk
Data Vault	GET	/api/data/export	Export all data as JSON
ML	POST	/api/ml/score	Score consent request (0-100)
ML	GET	/api/ml/health	ML service status and model info
Analytics	GET	/api/analytics	Dashboard metrics and charts
Audit	GET	/api/audit	Audit log (paginated)
Search	GET	/api/search	Global search across all records
User	PUT	/api/user/profile	Update profile and avatar
User	DELETE	/api/user/sessions/:id	Revoke active session

Security Controls

Control	Implementation
Authentication	JWT Bearer token required on all non-auth routes
Authorisation	Role-Based Access Control (user / admin middleware)
Password Storage	bcrypt with salt rounds = 10
Data Encryption	AES-256 for vault data at rest
HTTP Security	Helmet.js (CSP, HSTS, X-Frame-Options, CORS headers)
Session Tracking	IP address + User-Agent logged per login event
Consent Auto-Expiry	Cron job every 5 minutes auto-revokes expired consents

Control	Implementation
Two-Factor Auth	Time-limited OTP sent via email on login

7. Frontend Architecture

Page	Route	Description
Login	/login	JWT-based login with 2FA support
Signup	/signup	Registration with email verification
Verify OTP	/verify-otp	2FA OTP entry screen
Forgot Password	/forgot-password	Password reset request
Reset Password	/reset-password	Token-based password reset
Dashboard	/dashboard	Analytics, activity feed, KPI cards, charts
Data Vault	/data-vault	Personal data storage and file upload/view
Consent Hub	/consents	Full consent lifecycle management
Audit Logs	/audit	Immutable event ledger (paginated)
Settings	/settings	Profile, 2FA, sessions, export, delete account
Admin Users	/admin/users	User management - admin only

Key Features

- Real-time push notifications via WebSocket with unread badge count
- Live ML risk preview in New Consent form (debounced, non-blocking)
- Drag-and-drop file upload to Data Vault (PDF, DOC, images, CSV)
- File viewer opening files in browser with correct MIME type detection
- App / Website toggle in Consent Hub with clickable requester URL display
- Sortable paginated tables, dark-mode UI, Framer Motion animations
- Export personal data as JSON (GDPR Art. 20) and Delete Account flow

8. ML Risk Scoring System

ZeroShare includes a Python FastAPI microservice for consent risk scoring using a two-phase design that solves the cold-start problem.

Phase	Activation	Method	Benefit
Phase 1	Day 1 (zero data)	Rule-based heuristics	Works immediately
Phase 2	After 50+ decisions	Logistic Regression	Learns from admin decisions

Risk Features

Feature	Condition	Score
High-risk data type	passport, medical, SSN, biometric, genetic	+40
Medium-risk data type	financial, email, location, contacts, browsing	+20
Suspicious purpose	advertising, tracking, profiling, selling, sharing	+25
Vague purpose	Purpose statement shorter than 20 characters	+10
Long duration	1 year or more, or permanent / lifetime	+20
Medium duration	3 to 12 months access window	+10
Website requester	Requester type is website (not an app)	+5
Non-HTTPS URL	Website URL does not use HTTPS protocol	+10

Model Details

Property	Value
Algorithm	Logistic Regression — gradient descent, 1000 iterations, lr=0.1
Labels	GRANTED = safe (0), DENIED / REVOKED = risky (1)
Storage	JSON weights file — persisted via Docker named volume
Auto-Retrain	Every 24 hours via backend cron — needs 50+ samples
Fallback	Rule-based scoring when ML service unavailable

Sample API Response

```
{ "score": 72, "risk_level": "high", "confidence": "rule-based", "factors": ["Sensitive data type (identity/health)", "Purpose involves profiling", "Long access duration"] }
```

9. Email Notifications

Trigger Event	Email Subject
User Registration	Welcome to ZeroShare — Your data, your rules
Two-Factor Login	Your ZeroShare verification code
Password Reset	Reset your ZeroShare password
Consent Granted	Your consent request has been approved

Trigger Event	Email Subject
Consent Denied	Your consent request was denied
Consent Revoked	Data access has been revoked
Consent Expiring	Your consent expires in 7 days — action required
Consent Expired	Consent auto-revoked — access period ended

10. Docker Deployment

Container	Image	Port	Role
zeroshare_db	postgres:16-alpine	5432	PostgreSQL database
zeroshare_api	zeroshare-backend	5001	Node.js Express API + WebSocket
zeroshare_web	zeroshare-frontend	80	React SPA served by Nginx
zeroshare_ml	zeroshare-ml	8000	FastAPI ML risk scorer

Auto-Migration on Startup

On every backend startup, all ALTER TABLE ... IF NOT EXISTS migrations run automatically before the server accepts connections. Friends can run docker compose up --build with zero manual database setup steps.

Startup Sequence

1. PostgreSQL starts and passes health check (pg_isready)
2. Backend auto-runs all IF NOT EXISTS migrations
3. REST API starts on port 5001 + WebSocket server
4. Nginx serves React SPA on port 80, proxies /api/* to backend
5. ML service loads model weights (if any) — ready to score
6. Cron: consent auto-expiry every 5 minutes
7. Cron: ML retrain every 24 hours on real admin decisions

11. Design Decisions

Decision	Rationale
JWT over sessions	Stateless, scalable, works with WebSocket authentication
PostgreSQL over NoSQL	Structured relational data with referential integrity
AES-256 encryption	Industry standard for sensitive data at rest
FastAPI for ML	Python ML ecosystem, async performance, auto docs
Rule-based first	Solves cold-start: works day 1 with zero training data
Docker Compose	Zero-setup deployment — friends just run one command
Nginx proxy	Single port (80) in production, no CORS issues
WebSocket	Real-time UX without polling overhead
IF NOT EXISTS migrations	Auto-runs on startup — no manual steps needed

12. Privacy and Compliance

GDPR / DPDPA Principle	ZeroShare Implementation
Lawfulness and Transparency	Explicit consent with stated purpose per request
Purpose Limitation	Every consent requires a purpose field
Data Minimisation	Users choose exactly what data to store
Storage Limitation	Consent duration enforced with auto-expiry
Integrity and Confidentiality	AES-256 + Helmet.js security headers
Right to Access (Art. 15)	Full audit log viewable by user
Right to Portability (Art. 20)	Export all data as JSON from Settings
Right to Erasure (Art. 17)	Delete Account removes all data via CASCADE
Right to Object	Users can deny or revoke any consent anytime
Risk Transparency	ML risk score shown before granting consent

13. Future Enhancements

- End-to-end client-side encryption (zero-knowledge architecture)
- OAuth2 / OpenID Connect for third-party app authentication
- Automated GDPR and DPDPA compliance report generation
- Mobile application (React Native) for on-the-go management
- Blockchain-based immutable consent ledger
- Real app/website identity verification via third-party API
- Anomaly detection for suspicious data access patterns
- Multi-language (i18n) support

14. Project Statistics

Metric	Value
Frontend Pages	11
Backend API Modules	10 (auth, consents, data, analytics, audit, notifications, activity, search, user, ml)
Database Tables	7
Docker Services	4 (postgres, backend, frontend, ml-service)
Email Types	8
ML Risk Features	8
Languages	JavaScript (Node.js + React), Python, SQL
Lines of Code	8,000+
Repository	github.com/Ranjithhub08/ZeroShare

ZeroShare — Built with Privacy-by-Design principles. All features are fully implemented and deployed.